

Android XML 中 Material 组件与第三方库对比及选择

Android XML 中 Material 组件 & 替代第三方库

一、Material 组件（Material Components for Android, MDC）

核心定位

Google 官方推出的 **Material Design 风格 UI 组件库**，基于 AndroidX，是原生 UI 组件（如 Button、CardView）的增强版，内置 Material Design 设计规范（阴影、圆角、动画、配色），适配 Android 5.0+，无缝兼容 XML 布局开发。

核心优势

1. **官方维护**：与 Android 系统版本同步更新，兼容性、稳定性有保障；
2. **设计统一**：严格遵循 Material Design 3（最新版），自带“质感”（阴影层级、波纹动画、主题配色）；
3. **XML 友好**：所有组件支持 XML 直接声明，属性命名直观，无需额外代码；
4. **功能增强**：在原生组件基础上扩展实用功能（如按钮支持图标、卡片支持点击涟漪、输入框支持错误提示）。

常用核心组件（XML 示例）

1. 按钮（MaterialButton）

替代原生 Button，支持圆角、图标、主题色、波纹效果：

Code block

```
1 <com.google.android.material.button.MaterialButton
2     android:layout_width="wrap_content"
3     android:layout_height="wrap_content"
4     android:text="确认"
5     app:cornerRadius="8dp" <!-- 圆角 -->
6     app:icon="@drawable/ic_check" <!-- 图标 -->
7     app:backgroundTint="@color/primary" <!-- 背景色 -->
8     app:rippleColor="@color/white" <!-- 波纹色 -->/>
```

2. 卡片 (MaterialCardView)

替代原生 CardView，支持边框、点击阴影、圆角：

Code block

```
1  <com.google.android.material.card.MaterialCardView
2      android:layout_width="match_parent"
3      android:layout_height="wrap_content"
4      app:cardCornerRadius="12dp"
5      app:cardElevation="4dp"  <!-- 阴影高度 -->
6      app:cardStrokeWidth="1dp"  <!-- 边框宽度 -->
7      app:cardStrokeColor="@color/grey_200">
8
9      <!-- 卡片内部布局 -->
10     <TextView
11         android:layout_width="wrap_content"
12         android:layout_height="wrap_content"
13         android:text="Material 卡片"/>
14
15 </com.google.android.material.card.MaterialCardView>
```

3. 输入框 (TextInputLayout + TextInputEditText)

替代原生 EditText，支持 hint 浮动、错误提示、计数：

Code block

```
1  <com.google.android.material.textfield.TextInputLayout
2      android:layout_width="match_parent"
3      android:layout_height="wrap_content"
4      app:hint="请输入手机号"
5      app:errorEnabled="true"  <!-- 启用错误提示 -->
6      app:counterEnabled="true"  <!-- 启用计数 -->
7      app:counterMaxLength="11">
8
9      <com.google.android.material.textfield.TextInputEditText
10         android:layout_width="match_parent"
11         android:layout_height="wrap_content"
12         android:inputType="phone"/>
13
14 </com.google.android.material.textfield.TextInputLayout>
```

4. 其他常用组件

- `MaterialToolbar`：替代 ActionBar，支持自定义导航图标、菜单；

- `BottomNavigationView`：底部导航栏，自带选中动画；
- `Chip/ChipGroup`：标签组件（如“热门”“推荐”标签）；
- `Snackbar`：替代 Toast，支持操作按钮、滑动关闭。

集成方式（Gradle）

Code block

```
1 // 核心依赖（包含所有基础组件）
2 implementation 'com.google.android.material:material:1.12.0'
```

- 需在 `styles.xml` 中设置主题为 Material 主题（如 `Theme.Material3.DayNight.NoActionBar`）。

二、可替代的第三方 UI 库（XML 友好）

核心特点

第三方库专注于 **更丰富的 UI 样式、更简洁的 API、特定场景功能**，弥补 MDC 部分场景的灵活性不足（如复杂动画、自定义样式成本），以下为轻量、常用的选择：

1. ButterKnife（视图绑定，辅助 XML）

- **定位**：简化 XML 控件与 Java 代码的绑定（替代 `findViewById`），非 UI 组件库，但大幅提升 XML 开发效率；
- **核心优势**：注解式绑定，支持批量绑定、点击事件绑定，代码简洁；
- **使用示例**：

Code block

```
1 <!-- XML 中声明控件 -->
2 <TextView
3     android:id="@+id/tv_title"
4     android:layout_width="wrap_content"
5     android:layout_height="wrap_content"/>
```

Code block

```
1 // Java 中绑定（无需 findViewById）
2 @BindView(R.id.tv_title) TextView tvTitle;
3 @OnClick(R.id.btn_submit) void onSubmit() { ... }
4
```

```

5  @Override
6  protected void onCreate(Bundle savedInstanceState) {
7      super.onCreate(savedInstanceState);
8      setContentView(R.layout.activity_main);
9      ButterKnife.bind(this); // 绑定
10 }

```

- **集成：** `implementation 'com.jakewharton:butterknife:10.2.3'` + 注解处理器；
- **注意：** 仅支持 Java，Kotlin 推荐使用 ViewBinding（原生）。

2. SmartisanUI（锤子风格 UI，简洁美观）

- **定位：** 仿锤子手机的简约风格 UI 库，组件设计精致，支持 XML 直接使用；
- **核心组件：**
 - `SButton`：支持多种状态、渐变背景、图标位置自定义；
 - `SCardView`：更灵活的卡片（支持渐变背景、自定义阴影）；
 - `SEditText`：自带清除按钮、输入格式化；
- **优势：** 样式统一且美观，XML 属性丰富，无需额外自定义；
- **集成：** `implementation 'com.smartisanos:smartisan-ui:1.0.0'`。

3. XUI（全能型 UI 库，风格多样）

- **定位：** 支持 Material Design、iOS、微信等多种风格的 UI 库，组件齐全，XML 适配性强；
- **核心优势：**
 - 组件覆盖广（按钮、列表、弹窗、导航栏等）；
 - 支持主题切换，XML 属性可直接控制样式（无需代码干预）；
- **常用组件示例（XML）：**

Code block

```

1  <!-- 微信风格按钮 -->
2  <com.xuexiang.xui.widget.button.WeChatButton
3      android:layout_width="wrap_content"
4      android:layout_height="wrap_content"
5      android:text="微信风格按钮"
6      app:wx_button_type="primary"/>

```

- **集成：** `implementation 'com.xuexiang:xui:1.1.9'`（需配置主题）。

4. CircleImageView（圆形头像，单一功能）

- **定位：**专注于圆形/圆角头像的轻量级库，替代 MDC 的 `ShapeableImageView`（更简洁）；
- **核心优势：**API 极简，XML 直接声明，无需额外配置；
- **使用示例：**

Code block

```
1 <de.hdodenhof.circleimageview.CircleImageView
2     android:layout_width="80dp"
3     android:layout_height="80dp"
4     android:src="@drawable/avatar"
5     app:civ_border_width="2dp"  <!-- 边框宽度 -->
6     app:civ_border_color="@color/white"/>  <!-- 边框颜色 -->
```

- **集成：** `implementation 'de.hdodenhof:circleimageview:3.1.0'`。

5. MaterialishProgress（进度条增强）

- **定位：**Material 风格的进度条扩展库，弥补 MDC 进度条样式单一的问题；
- **核心组件：**
 - `CircleProgress`：圆形进度条（支持渐变、动画速度调整）；
 - `LinearProgress`：线性进度条（支持圆角、渐变）；
- **使用示例（XML）：**

Code block

```
1 <me.zhanghai.android.materialprogressbar.MaterialProgressBar
2     android:layout_width="wrap_content"
3     android:layout_height="wrap_content"
4     app:mpb_progressStyle="circular"
5     app:mpb_tint="@color/primary"  <!-- 进度条颜色 -->
6     app:mpb_indeterminate="true"/>  <!-- 无限循环动画 -->
```

- **集成：** `implementation 'me.zhanghai.android.materialprogressbar:library:1.6.1'`。

三、选择建议

1. **优先用 MDC：**如果需求是“基础 UI + 符合 Material Design 规范”，MDC 是首选（官方维护、无兼容性风险）；
2. **选第三方库的场景：**

- 需要特定风格（如锤子、微信）→ SmartisanUI/XUI；
- 单一功能强化（如圆形头像、进度条）→ CircleImageView/MaterialishProgress；
- 简化 XML 绑定 → ButterKnife（Java）/ViewBinding（Kotlin）；

3. **注意：**第三方库尽量“少而精”，避免引入过大的库导致 APK 体积增加。

（注：文档部分内容可能由 AI 生成）